Clock-Sensitive Waits and Fault-Aware Repair in Zephyr RTC Tests

Pranav Krishnan
The University of Texas at Austin
Austin, Texas, USA

Keshav Bhargava
The University of Texas at Austin
Austin, Texas, USA

ABSTRACT

Firmware tests often wait a fixed time for a hardware event. A legal but slower clock can make a correct program fail that test. We study clock-sensitive waits and fault-aware repair using one Zephyr STM32 real-time clock (RTC) driver and the existing Renode model. Three selected upstream cases and three generated pending suites fail at legal clock settings. Each of four mechanical repairs makes each subject pass all 102 tested legal settings while retaining its measured slow-oscillator fault detection. This six-subject result is separate from right-sizing overlong deadlines in 11 already clock-tolerant generated counting tests, which adds slow-fault detection without requiring state polling. Added alarm checks address another weakness: early and late alarm programming. An expanded full-application study contains 10 distinct runnable upstream case names, of which four have waiting-sensitive failures; only the original three upstream subjects were repaired. We distinguish timing sensitivity across settings from unmeasured same-setting flakiness and separate waiting defects, cleanup cascades and unresolved rate contracts. The results support these repairs under the tested model and faults, not a general repair rate or a prevalence estimate for developer tests.

KEYWORDS

embedded testing, timing sensitivity, test repair, RTC, fault detection

1 INTRODUCTION

An RTC test schedules an alarm ten calendar seconds ahead, waits thirteen simulated (Renode virtual) seconds, and asserts that a callback arrived. Its wait assumes that the RTC clock and callback path fit the observation window. At a slower but datasheet-legal clock, the alarm target itself may occur after the assertion. Extending the wait can fix the legal-clock failure. An excessively long deadline can also accept a faulty clock, so a passing repaired test alone is insufficient.

Our question is: **when legal clock variation changes a test verdict, can we repair the timing assumption without weakening measured fault detection?** We analyze the completed paired repair records for one Zephyr STM32 RTC driver in Renode. The main result concerns six subjects, shown individually in Table 1. We separate their repair from correcting over-padding in other subjects, and distinguish deadline sizing, state polling and additional alarm checks.

Connection to flaky tests. Timing assumptions can make tests unreliable across operating conditions [4, 5, 8]. Here, a deterministic pass at one fixed frequency and failure at another is *timing sensitivity*, not observed nondeterminism at one setting.

Virtual time (s)	One upstream callback iteration
0	Set RTC time; alarm is ten calendar seconds ahead.
3	Check that no callback has arrived early.
13	Original final check: callback required.
18.824	Ideal target arrival at legal 17 kHz.
20.824	Repaired deadline: require callback by here.

Figure 1: Why the 13-second wait fails. Times illustrate zero initial subsecond phase and omit delivery latency; they are not measured arrival times.

We did not measure same-setting flakiness or a device-frequency population distribution. Repeated nominal screens and attribution replays do not replace a systematic fixed-frequency repeatability experiment. Nor are selected developer tests a random sample from which to estimate flaky-test prevalence.

2 APPROACH: REPAIR THE WAIT, RECHECK THE ORACLE

2.1 A clock-derived deadline

The pinned Zephyr v4.3.0 source [9] selects the low-speed internal oscillator (LSI) for the STM32F4 Discovery RTC. DS8626 Rev 12 characterizes its frequency as 17–47 kHz at 3 V and −40 to 105 °C [7]. With a fixed 128×250 prescaler, ten RTC seconds ideally take

$$t(f) = 320{,}000/f \quad \text{simulated seconds, with } f \text{ in Hz.}$$

At 17 kHz this is 18.824 s, beyond the original 13-second wait. Its ideal boundary is 24.615 kHz. These are manually source-derived thresholds, not an automatically inferred specification; full-application phase effects can require a band rather than one boundary.

The repair ceiling is **20.824 simulated seconds**: the ideal slow-clock duration, 18.823529… s, rounded up to the next millisecond (18.824 s), plus **two seconds of unvalidated experimental allowance**. It is not a measured or guaranteed interrupt-to-callback latency bound. Kernel uptime measures the wait while CPU/SysTick remain fixed. Figure 1 makes the original waiting assumption concrete.

In the upstream callback source, lines 120 and 132 sleep for constants of three and ten seconds; lines 126–129 check for premature callbacks and lines 140–141 require arrival. V1/V1b retain the early check and sleep only until the revised deadline; V2 can finish when the completion predicate becomes true. At the injected 10-kHz slow fault, the same target ideally takes 32 s, so both the original and repaired deadlines reject it. This explains why extending this subject's wait can preserve its slow-fault check.

2.2 Paired mechanical variants

V0 is the original test. **V1**, naive extension, uses the larger of the original wait and 20.824 s; it never shortens an overlong wait. **V1b**, right-sizing, uses a 20.824-second total ceiling while preserving the original fixed-sleep or polling strategy and cadence. **V2** uses the same ceiling and original completion predicate with 20-ms state polling, latching destructive pending reads. **V3** adds missing checks from the frozen task specification: supported alarm-field readback, callback identity/calendar checks, or counting progress/monotonicity as applicable. This deliberately augments upstream oracles. Original targets, early checks and non-time assertions remain. V1 preserves existing service stages (Sol pending r03 can reach 23.824 s); V1b–V3 share one total ceiling. The dated transformation rules were fixed before repair builds.

Faults are tested separately: fast 64-kHz and slow 10-kHz oscillator inputs, a nominal-clock *nonarrival fault* suppressing alarm delivery or freezing counting reads, and nominal-clock alarm-compare offsets of −1 or +1 RTC second. A table *kill* means an activated fault detected by an explicit selected-case assertion. Alarm programming, calendar-target arrival and callback delivery are different properties: readback checks programming; calendar equality also assumes delivery before the calendar advances. No delivery-delay experiment was performed.

3 EXPERIMENTAL SETUP AND SCOPE

Scope. The original comparison purposively selected three upstream cases for visible timing windows: alarm callback, alarm pending and calendar counting. The later full-application expansion contains **10 distinct runnable upstream case names**, including those three, from 19 declared names: five runnable RTC API cases, four counter basic cases and one counter seconds case. Four distinct names have waiting-sensitive failures; only the original three upstream subjects were repaired. Configurations share **one Renode STM32F4 RTC model**, so board labels do not provide independent hardware replication. Counter applications use **non-stock RTC-only configurations**, with the existing counter adapter enabled and other timer counters and the RTC API driver disabled; they contextualize scope, not a second repaired driver. Capability predicates are retained. Neither this selection nor the repair subset estimates developer-test prevalence.

Execution. Renode v1.17.0 is an instruction-level embedded simulator [1]. Each run starts a fresh process with unchanged source/ELF, CPU, SysTick and synchronization quantum. The frozen schedule has 100 shared integer-frequency settings plus two endpoints within the documented interval. Build, five nominal passes and small calibration precede production; the Linux container was calibrated against local verdicts and trace hashes. All records analyzed here already existed; no study was rerun for this revision.

F4, F767 and F746 use 17–47 kHz; F767 is explicitly an F4 configuration replica. H753 uses the frozen DS12117 Rev 10 revision-V envelope, 29.760–33.600 kHz at VDD 1.62–3.6 V and junction −40 to 105 °C [13]. F746 counter basic is excluded after an MPU fault, F072 after no test output, and watchdog after no demonstrated reboot. The F072 divider observation remains unconfirmed.

Attribution and missingness. Same-input failure replays, exact source lines and task-specific references distinguish waiting defects, phase-dependent waits, cleanup cascades and unresolved rate contracts. References check premature behavior and progress under their registered ceiling; a passing progress reference cannot override a failed rate reference. Capability skips and missing verdicts remain separate from passes. Infrastructure-interrupted evaluations without a verdict were resumed under a dated amendment, while finalized harness-error outcomes remain missing.

Repair cohort. The roster was fixed after the initial clock study and before repair builds: three original isolated upstream cases plus 30 generated scalar-budget suites (27 counting, three pending), with no generated callback suite. The expanded full applications were not repaired. This selection favors identifiable waiting budgets and is not a general repair-success sample. Each variant has 3,300 seeded legal slots and 66 endpoints; alarm offsets apply to five alarm subjects. Paired comparisons use only subjects evaluable in both variants.

4 RESULTS

4.1 Six sensitive subjects: repaired waits retain slow-fault detection

Each of the six subjects passes the tested legal settings after every repair and retains its measured slow-fault detection. Table 1 shows the three upstream subjects and three generated pending suites separately. Every repaired subject passes all 102 legal settings with zero missing verdicts. Naive extension already achieves this result: it does *not* weaken slow-fault detection in these six subjects.

As supporting detail, the six subjects contribute 612 shared subject-setting pairs: V0 produces 500 passes and 112 failures; each repair produces 612 passes. These are **six subjects, not 612 independent tests**. Their repeated clock settings describe each subject's sensitivity. The 112 original failing slots have passing task-specific references and become observed passes under all repairs. This result concerns the original isolated contexts, not all expanded case names or untested clock settings.

4.2 A separate result: right-sizing over-padded counting tests

Right-sizing adds 11 slow-fault detections in different, already clock-tolerant generated counting subjects. Across the full 33-subject cohort, V1 retains V0's 21/33 slow-fault detections because it preserves long waits. V1b detects 32/32 evaluable slow faults: the 32 common V1/V1b pairs retain 21 detections, gain 11 and lose none. All 11 gains are generated counting subjects with 102 V0 legal passes and no missing legal verdicts. They correct over-padding; they are *not* the six sensitive subjects in Table 1.

V0 has 106 seeded and six endpoint failures. Every repair has zero observed legal failures across the cohort, but V1/V1b/V2/V3 retain respectively 7/5/3/2 missing seeded outcomes; all 66 endpoints are observed passing. There are 29 finalized missing outcomes across legal and fault probes. They are not counted as successes. All 458 primary assertion failures replay exactly, including raw trace hashes.

Table 1: The six originally sensitive subjects. Each repair is compared with the same subject's V0 records. All six detect the slow fault in V0 and each repair, with no missing outcomes.

	V0 failures		Legal passes / 102 settings				Slow fault
Subject	Seeds / 100	Ends / 2	V1	V1b	V2	V3	V0 → each repair
Upstream, isolated cases							
Alarm callback	26/100	1/2	102/102	102/102	102/102	102/102	detect → detect
Alarm pending	26/100	1/2	102/102	102/102	102/102	102/102	detect → detect
Calendar counting	17/100	1/2	102/102	102/102	102/102	102/102	detect → detect
Generated, base prompt							
Sol pending r01	11/100	1/2	102/102	102/102	102/102	102/102	detect → detect
Sol pending r03	13/100	1/2	102/102	102/102	102/102	102/102	detect → detect
Astra pending r05	13/100	1/2	102/102	102/102	102/102	102/102	detect → detect

Table 2: Full 33-subject cohort. Seed F/M: failures/missing out of 3,300 slots. Fault cells: detections/evaluable. Median is whole-test virtual time.

Variant	Seed F/M	Slow	Early	Late	Median (s)
V0	106/9	21/33	4/5	4/5	11.0
V1	0/7	21/33	4/5	4/5	11.0
V1b	0/5	32/32	4/5	4/5	11.0
V2	0/3	33/33	4/5	4/5	11.0
V3	0/2	33/33	5/5	5/5	11.0

Table 3: Distinct case names per configuration. Run: ever runnable; Skip: always capability-skipped. Wait, Cascade and Rate are separate failure categories.

Configuration	Run	Skip	Wait	Cascade	Rate
F4	10	9	4	1	1
F767 (F4 replica)	10	9	4	1	1
F746	6	2	3	0	1
H753	10	9	0	0	0

4.3 Polling and added alarm checks do different work

Polling adds no paired slow-fault detection, while extra alarm checks detect a previously accepted programming error. V2 detects the slow fault in 33/33 subjects, but all 32 common V1b/V2 pairs already detect it under V1b. The remaining V1b outcome is missing, so the denominator change supplies no evidence of a polling gain. Polling instead shortens the upstream callback's median seeded whole-test duration from 42.0 s (V1b) to 20.75 s (V2). The pooled median is 11.0 s for every variant and hides this difference; whole-test time is not callback latency.

V0–V2 detect each alarm-offset sign in 4/5 applicable subjects. V3 reaches 5/5 for each sign by adding detection in the upstream callback. Readback and callback-calendar checks were introduced together; these records cannot isolate timestamp checking's effect. Across adjacent variants, no detected fault becomes accepted on common-evaluable subject pairs. Nonarrival is detected in 32/32 evaluable V0 subjects and 33/33 after repair; its missing V0 verdict is not a paired gain. No evaluable fast-oscillator probe is detected (0/32 for V1b, 0/33 otherwise). Preservation of this measured fault set therefore does not establish a complete oracle.

4.4 Expanded applications: scope and failure attribution

Four waiting-sensitive names do not mean that every failing case is an independent test defect. Table 3 separates waiting failures from cleanup cascades and unresolved rates. Across 11 application/configuration combinations, 1,122 launches produce 3,022 passing, 470 failing and 3,138 capability-skipped case slots, with no missing primary case verdicts. Per board, waiting-sensitive names are 4/10 runnable names on F4 and its F767 replica, 3/6 on F746, and 0/10 on H753. These are within-cohort fractions, not population prevalence estimates.

Pending has the ideal 24.615-kHz boundary. F4 full-application callback failures span sampled 17.000–23.673 kHz (passes 23.847–47.000); counting failures span 17.000–20.500 (passes 20.701–47.000). The counter all-channels wait has a source-derived phase envelope of 21.333–32.000 kHz. These phase-dependent cases cannot all be summarized by one exact frequency threshold: writing the RTC calendar preserves the subsecond count in the existing model. Relative to isolated runs, F4 full/isolated seeded failure counts are pending 26/26, callback 22/26 and counting 12/17. Phase carryover and ordering are a source-supported explanation, *not directly measured phase.*

A later counter single-shot setup failure follows skipped cleanup and is a cascade, not a fifth independent waiting-sensitive name. Counter seconds fails over sampled 17.000–26.168 kHz and passes 27.086–47.000, but its 105 rate references also fail while progress checks pass. It remains a firmware/simulator/accuracy-contract investigation, not a demonstrated waiting defect. All 470 failing slots have reference outcomes: 277 waiting references pass, 88 clean-state cascade references pass and 105 rate references fail. H753 has no observed failures in its narrower modeled interval; one counter case skips nominally but runs at some settings. This contrast varies the range/configuration within the shared model, not silicon validation. Source excerpts and complete per-case bands remain in the artifact.

4.5 Secondary generated-test context

The generated cohort covers only the original three task purposes: `gpt-6-luna`, `gpt-6-sol` and `gpt-6-astra` each produced five source-only ztests (Zephyr's test framework) per task and prompt. Of 90 attempts, 66 pass functional/build/five-nominal gates. The base prompt omits numeric clock ranges; the other adds ranges and a delivery reminder. Suite-level clock sensitivity is 3/34 base versus 0/32 with ranges (two-sided Fisher $p = 0.239$); 12 original seeded verdicts remain missing. Models received purpose-derived specifications, not upstream code or execution feedback. The specifications requested stronger observables, including configured-time callbacks/IDs and pending readback/not-before checks. With no same-spec human baseline, neither oracle differences nor this small prompt contrast establishes model superiority or an isolated numeric-range effect. Detailed generation, yield, two-axis, fault and distribution tables are retained in the artifact rather than expanding this comparison here.

5 DISCUSSION AND LIMITATIONS

What the repair evidence supports. The six-subject result shows that these waits can accept tested legal clocks while retaining measured slow-fault detection. The different 11-subject counting result shows why preserving an already overlong wait can leave a weak slow-clock oracle. Right-sizing supplies those gains without polling; polling reduces time; augmented checks improve alarm-offset detection. None establishes semantic equivalence for every defect or an automatic general repair procedure.

Clock ranges and distributions. The pinned HSI (high-speed internal oscillator) context study varied 15.84–16.16 MHz (±1% at 25 °C), with integer 99–101-MIPS CPU mapping: no failures among 943 upstream and 3,270 generated evaluable seeded runs; 87 seeded outcomes were not evaluable. Different tasks, the narrow range and throughput approximation prevent a direct HSI-versus-LSI causal comparison. The RTC study isolates LSI variation with fixed CPU/SysTick; its 17–47-kHz range is much wider. The HSI result does not imply HSI can never matter.

Neither uniform sampling nor a datasheet interval measures how frequencies are distributed across devices. We lead with boundaries/bands, retaining uniform and legal-conditioned normal sensitivity calculations in the artifact as illustrative assumptions only. ST documents describe oscillator accuracy/calibration, not a usable F407 population distribution for this analysis [10–12]. The board survey's 106 LSI versus 40 LSE (external-crystal) RTC-enabled targets supplies context, not a flaky-test prevalence estimate.

Practical guidance and limits. Derive waits from the worst-case legal clock, justify a delivery allowance, retain early-arrival checks against an independent time reference, and assert the RTC calendar/programming target rather than eventual arrival alone. Our extra two seconds is an unvalidated experimental allowance: no explicit interrupt-to-callback latency bound was found in 33 scanned files. A future delivery bound cannot be inferred from these records. Fast-fault blindness, purposive selection, shared model behavior and non-stock counters limit external validity. Source/padding coding used one unblinded reviewer; related suites and shared frequencies are not independent replication. The artifact's 2× padding cutoff is descriptive, not a correctness threshold. Missing verdicts remain missing.

6 RELATED WORK

Flaky-test studies report asynchronous waits and environment assumptions [4, 8]; FlakeRake perturbs timing to reproduce intermittent failures [5]. Our clock changes expose sensitivity across settings without demonstrating within-setting nondeterminism. Interrupt-driven testing likewise explores scheduling conditions [6]. Mutation testing and oracle research motivate fault checks after a repair, rather than accepting nominal passes alone [2, 3]. Our contribution is the paired separation of wait repair, deadline sizing and oracle augmentation in this RTC setup.

7 CONCLUSION

Each of six originally sensitive subjects passes its tested legal settings after repair and retains slow-fault detection. Separately, right-sizing improves detection in 11 already clock-tolerant counting subjects. Polling and alarm checks provide different benefits. These results support fault-aware repair of selected clock-sensitive waits, with explicit limits on subjects, faults and simulator scope.

AI USE

LLMs generated the evaluated suites. Automated tooling assisted execution, auditing and manuscript preparation under author direction; the authors review the evidence and take responsibility for the claims.

CODE AND DATA AVAILABILITY

Code, frozen tests, amendments and data dictionary; results. The unchanged evidence release v1 contains an *older PDF*. The current PDF, TeX and claim audit are separately versioned as manuscript v2. `make tables` verifies the pinned archive and rebuilds its eight detailed tables without Renode; the companion script generates this paper's three readable table views from those same records.

REFERENCES

[1] Antmicro. n.d. Renode time framework documentation. Renode documentation (accessed 2026-11-04). https://renode.readthedocs.io/en/latest/advanced/time_framework.html

[2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5, 507–525. https://doi.org/10.1109/TSE.2014.2372785

[3] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5, 649–678. https://doi.org/10.1109/TSE.2010.62

[4] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 643–653. https://doi.org/10.1145/2635868.2635920

[5] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 269–280. https://doi.org/10.1109/ICST60714.2024.00032

[6] John Regehr. 2005. Random Testing of Interrupt-Driven Software. In *Proceedings of the 5th ACM International Conference on Embedded Software*, 290–298. https://doi.org/10.1145/1086228.1086282

[7] STMicroelectronics. 2026. STM32F405xx, STM32F407xx Datasheet. STMicroelectronics datasheet DS8626 Rev 12. https://www.st.com/resource/en/datasheet/stm32f407vg.pdf

[8] P. E. Strandberg, T. J. Ostrand, E. J. Weyuker, W. Afzal, and D. Sundmark. 2020. Intermittently Failing Tests in the Embedded Systems Domain. In *Proceedings of*

the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, 337–348. https://doi.org/10.1145/3395363.3397359

[9] Zephyr Project. 2025. Zephyr RTOS v4.3.0 (release commit 3568e1b6d5cd). Zephyr Project source revision. https://github.com/zephyrproject-rtos/zephyr/commit/3568e1b6d5cdd51a6b964a2a1d6d29200fea2056

[10] STMicroelectronics. 2018. How to optimize STM32 MCUs internal RC oscillator accuracy. Application note AN5067, Rev 1. https://www.st.com/resource/en/application_note/an5067-how-to-optimize-stm32-mcus-internal-rc-oscillator-accuracy-stmicroelectronics.pdf

[11] STMicroelectronics. 2023. How to calibrate internal oscillators on STM32G0 MCUs. Application note AN5126, Rev 2. https://www.st.com/resource/en/application_note/an5126-how-to-calibrate-internal-oscillators-on-stm32g0-mcus-stmicroelectronics.pdf

[12] STMicroelectronics. 2024. STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs. Reference manual RM0090, consulted Rev 21, Sec. 6.2.11. https://www.st.com/resource/en/reference_manual/rm0090-stm32f4xx-reference-manual-stmicroelectronics.pdf

[13] STMicroelectronics. 2026. STM32H753xI Datasheet. DS12117, Rev 10, Tables 38 and 135; registered revision V envelope. https://www.st.com/resource/en/datasheet/stm32h753zi.pdf